

RELAZIONE TECNICA DI PROGETTO

Progetto: Analizzatore parametri di rete | **Autore:** Gabriele Boscarello

Linguaggio Principale: Python 3

Interfaccia Grafica: Tkinter (ttk - clam theme)

Ambiente di Test: Laboratorio Isolato

Libreria Sniffing: Scapy

1. Descrizione Generale

Analizzatore parametri di rete è un'applicazione desktop sviluppata in Python che offre strumenti di diagnostica e monitoraggio per reti locali in ambienti di laboratorio controllati. L'obiettivo principale è fornire un'interfaccia grafica intuitiva e moderna per due attività fondamentali:

- Scansione porte e host:** rileva la presenza di servizi attivi, valutando lo stato dei socket e la latenza in millisecondi.
- Live Packet Sniffing:** ispeziona il traffico dati in tempo reale, decodificando orari, IP sorgente/destinazione, protocolli e dimensioni.

2. Architettura e Moduli Principali

L'applicazione è strutturata attorno alla classe NetworkAnalyzerApp (Tkinter con tema ttk 'clam' in Dark Mode):

A. Port & Host Scanner

- Socket Programming & Latenza:** Crea connessioni TCP (SOCK_STREAM) con timeout a 150ms e misura il tempo di handshake con il modulo time.
- Mappatura Servizi:** Risolve i nomi dei servizi attivi via socket.getservbyport() (es. HTTP, Telnet, Domain).

B. Live Packet Sniffer

- Integrazione Scapy:** Esegue la cattura ed il parsing del traffico a livello di rete (IP) e trasporto (TCP, UDP, ICMP).
- Parsing Protocolli:** Estrae orario, IP sorgente/destinazione, porte e dimensione del pacchetto in Byte.

3. Architettura Multithread

Per evitare il blocco dell'interfaccia utente (UI Freeze) durante operazioni d'I/O bloccanti, il software adotta un'architettura Multithread:

1. Thread UI (Principale)	Gestisce il rendering dell'interfaccia Tkinter e gli eventi dei pulsanti.
2. Background Threads	Esegue la scansione ed il packet sniffing in thread separati (threading).
3. Sincronizzazione UI	Aggiorna la GUI in modo thread-safe tramite il metodo root.after().

4. Requisiti e Permessi

Richiede Python 3.x, le librerie standard (socket, threading, tkinter, time) e il modulo esterno scapy.

⚠ NOTA DI SICUREZZA: Lo sniffing richiede modalità promiscua e privilegi di amministratore (sudo su macOS/Linux). I test sono stati condotti esclusivamente in un ambiente di laboratorio isolato.

📝 Nota sul Documento: La presente relazione tecnica è stata redatta e strutturata con il supporto dell'Intelligenza Artificiale per l'ottimizzazione della documentazione. Il codice sorgente dell'applicazione, la sua logica di funzionamento e l'architettura del software sono stati interamente ideati e sviluppati dall'autore Gabriele Boscarello.